

WEEK 2: PRODUCT AND CATEGORY MANAGEMENT SYSTEM

- HEMAVATHI.B(E25AI114)

Scenario

The Perfume shop company wants to maintain information about different Brands and their categories. Each Brand belongs to a particular category. The system should store and manage details such as **Brand Name, Price, and Stock Availability**.

1. Design Brand and Category Tables

Category Table

The Category table is used to store information about different Brand categories. The **Category_ID** uniquely identifies each category.

```
SQL> CREATE TABLE Category (Category_ID VARCHAR(10) PRIMARY KEY, Category_Name VARCHAR(30) NOT NULL);  
Table created.
```

Brand Table

The Brand table is used to store details about the Brands. It contains information such as **Brand_ID, Brand_Name**.

```
SQL> CREATE TABLE Brand(Brand_ID VARCHAR(10) PRIMARY KEY, Brand_Name VARCHAR(30) NOT NULL);  
Table created.
```

The two tables, **Category** and **Brand**, have been created. Now, we need to define the relationship between the tables and apply the required constraints.

2. Define Relationships and Constraints

Relationship Between Category and Brand

A category can contain many brand, but each Brand belongs to only one category. Therefore, the relationship between the two tables is:

One Category → Many Brand

**Relationship: One-to-Many (1 :M)*

Primary Key

A **Primary Key** is a column that uniquely identifies each record in a table. It does not allow duplicate values.

- **Category_ID** is the Primary Key in the Category table.

- **Brand_ID** is the Primary Key in the Brand table.

```
SQL> ALTER TABLE Product ADD CONSTRAINT PK_product PRIMARY KEY(product_ID);  
Table altered.
```

Foreign Key

A **Foreign Key** is used to connect two tables.

```
SQL> ALTER TABLE Inventory  
  2 ADD CONSTRAINT fk_inventory_brand  
  3 FOREIGN KEY (Brand_ID) REFERENCES Brand(Brand_ID);  
  
Table altered.
```

The **Brand_ID** in the Brand table acts as the Foreign Key. It refers to the **Brand_ID** in the Brand table.

NOT NULL Constraint

The **NOT NULL** constraint is used when a column must contain a value. It does not allow the field to be left empty.

CHECK Constraint

The **CHECK** constraint is used to make sure that the value entered satisfies a particular condition.

For example, the Brand price can be checked to make sure that it is greater than zero.

```
SQL> ALTER TABLE Brand ADD CONSTRAINT CHK_brand_Name CHECK (LENGTH(TRIM(BRAND_NAME))>0);  
Table altered.
```

3. Insert Values

Category Table

Values are inserted into the Category table using the **INSERT** command.

```
SQL> INSERT INTO Category VALUES ('&Category_ID', '&Category_Name');
Enter value for category_id: CAT-010
Enter value for category_name: Luxury
old 1: INSERT INTO Category VALUES ('&Category_ID', '&Category_Name')
new 1: INSERT INTO Category VALUES ('CAT-010', 'Luxury')

1 row created.
```

```
SQL> CREATE TABLE Category (Category_ID VARCHAR(10) PRIMARY KEY, Category_Name VARCHAR(30) NOT NULL);
Table created.

SQL> INSERT INTO Category VALUES('CAT-001', 'Floral');
1 row created.

SQL> INSERT INTO Category VALUES('CAT-002', 'Woody');
1 row created.

SQL> INSERT INTO Category VALUES('CAT-003', 'Fresh');
1 row created.

SQL> INSERT INTO Category VALUES('CAT-004', 'Oriental');
1 row created.

SQL> INSERT INTO Category VALUES('CAT-005', 'Citrus');
1 row created.

SQL> INSERT INTO Category VALUES('CAT-006', 'Fruity');
1 row created.

SQL> INSERT INTO Category VALUES('CAT-007', 'Vanilla');
1 row created.

SQL> INSERT INTO Category VALUES('CAT-008', 'Musky');
```

Brand Table

Values are inserted into the Brand table using the **INSERT** command.

BRAND_ID	BRAND_NAME	CATEGORY_I	STOCK_QUANTITY	PRICE
PRODUCT_COUNT				
BR-01	Dior 1	CAT-001	25	1500
BR-02	Chanel 1	CAT-002	30	1800
BR-03	Gucci 1	CAT-003	20	1200
BRAND_ID	BRAND_NAME	CATEGORY_I	STOCK_QUANTITY	PRICE
PRODUCT_COUNT				
BR-04	Versace 1	CAT-004	15	2200
BR-05	Armani 1	CAT-005	35	1400
BR-06	Burberry 1	CAT-006	28	1600
BRAND_ID	BRAND_NAME	CATEGORY_I	STOCK_QUANTITY	PRICE
PRODUCT_COUNT				
BR-07	Calvin	CAT-007	10	3500

CRUD Operations

CRUD stands for:

- **C – Create**
- **R – Read**
- **U – Update**
- **D – Delete**

CRUD operations are used to manage the records stored in the database.

Insert Record

A new Brand can be added to the Brand table using INSERT command

```
SQL> INSERT INTO Brand (BRAND_ID,BRAND_NAME,CATEGORY_ID,STOCK_QUANTITY,PRICE,PRODUCT_COUNT) VALUES ('BR-11','Park Avenue','CAT-011',20,1800,4);  
1 row created.
```

Update Price

The price of an existing Brand can be changed using the **UPDATE** command.

```
SQL> UPDATE Brand SET PRICE = 2000 WHERE BRAND_ID = 'BR-11';  
1 row updated.
```

Delete a Brand

An existing Brand can be removed from the Brand table using the **DELETE** command.

```
SQL> DELETE FROM Brand WHERE Brand_ID='BR-08';  
1 row deleted.
```

4. Generate Category-wise Brand Reports

1. Display all Brands under each category

This report is used to display all the Brands belonging to each category.

```
SQL> SELECT c.CATEGORY_NAME,b.BRAND_NAME,b.PRODUCT_COUNT,b.STOCK_QUANTITY,b.PRICE FROM Category c JOIN Brand b ON c.CATEGORY_ID=b.CATEGORY_ID ORDER BY c.CATEGORY_NAME;
```

CATEGORY_NAME	BRAND_NAME	PRODUCT_COUNT
Aquatic	YSL	1
Citrus	Armani	1
Floral	Dior	1
Fresh	Gucci	1
Fruity	Burberry	1
Luxury	Fogg	1

CATEGORY_NAME	BRAND_NAME	PRODUCT_COUNT
Aquatic	YSL	1
Citrus	Armani	1
Floral	Dior	1
Fresh	Gucci	1
Fruity	Burberry	1
Luxury	Fogg	1

2. Count the number of Brands in each category

This report shows how many Brands are available under each category.

```
SQL> SELECT c.CATEGORY_NAME, COUNT(b.BRAND_ID) AS PRODUCT_COUNT FROM Category c JOIN Brand b ON c.CATEGORY_ID=b.CATEGORY_ID GROUP BY c.CATEGORY_NAME ;
```

CATEGORY_NAME	PRODUCT_COUNT
Rusky	1
Floral	1
Oriental	1
Aquatic	1
Citrus	1
Vanilla	1
Luxury	1
Woody	1
Fresh	1
Fruity	1

10 rows selected.

3. Find the highest-priced Brand in each category

This report is used to find the Brand with the highest price in every category.

```
SQL> SELECT c.CATEGORY_NAME, b.BRAND_NAME, b.PRICE FROM Category c JOIN Brand b ON c.CATEGORY_ID=b.CATEGORY_ID WHERE b.PRICE=(SELECT MAX(b2.PRICE) FROM Brand b2 WHERE b2.CATEGORY_ID=b.CATEGORY_ID);
```

CATEGORY_NAME	BRAND_NAME	PRICE
Floral	Dior	1500
Woody	Chanel	1800
Fresh	Gucci	1200
Oriental	Versace	2200
Citrus	Armani	1000
Fruity	Burberry	1600
Vanilla	Calvin	3500
Aquatic	YSL	2500
Luxury	Fogg	2000
Musky	Hugo Boss	1900

10 rows selected.

4. Display categories having more than 5 Brands

This report displays only those categories that contain more than five Brands.

```
SQL> SELECT c.CATEGORY_NAME, COUNT(b.BRAND_ID) AS PRODUCT_COUNT FROM Category c JOIN Brand b ON c.CATEGORY_ID=b.CATEGORY_ID GROUP BY c.CATEGORY_NAME HAVING COUNT(b.BRAND_ID)>5;
```

no rows selected

5. Find the average Brand price category-wise

This report calculates the average price of Brands available in each category.

```
SQL> SELECT c.CATEGORY_NAME, AVG(b.PRICE) AS AVERAGE_PRICE FROM Category c JOIN Brand b ON c.CATEGORY_ID=b.CATEGORY_ID GROUP BY c.CATEGORY_NAME;
```

CATEGORY_NAME	AVERAGE_PRICE
Musky	1900
Floral	1500
Oriental	2200
Aquatic	2500
Citrus	1400
Vanilla	3500
Luxury	2000
Woody	1800
Fresh	1200
Fruity	1600

10 rows selected.
